中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 蓝宇舟 学号 2022K8009918005 专业 计算机科学与技术
实验项目编号 1 实验名称 基本功能部件——寄存器堆和算术逻辑单元

一、 逻辑电路结构与仿真波形的截图及说明

1 寄存器堆设计

```
// reg file
reg [`DATA_WIDTH - 1:0] my_reg_file [`REG_NUM - 1:0];

always @(posedge clk) begin
    // when to accept input
    if (wen && (waddr != `ADDR_WIDTH'b0)) begin
        my_reg_file[waddr] <= wdata;
    end
end
// read data from specific address
assign rdata1 = {`DATA_WIDTH{raddr1 != `ADDR_WIDTH'b0}} & my_reg_file[raddr1];
assign rdata2 = {`DATA_WIDTH{raddr2 != `ADDR_WIDTH'b0}} & my_reg_file[raddr2];
```

图 1: 寄存器堆关键代码截图

图 1 是寄存器堆的关键代码截图, 我定义了一个名为 my_reg_file 的 reg 型变量, 用于存储 32 个 32 位寄存器。

写入操作是同步写入, 为时序逻辑。在 always 块中, 敏感列表为 clk 信号, 根据输入的地址和数据, 对寄存器堆进行写操作。当写使能信号为 1 时, 将数据写入到指定地址的寄存器中。这里注意不能向 0 号寄存器写入数据。

读取操作是异步读出, 为组合逻辑。这里我用两个 assign 语句实现了读取操作。将指定地址的寄存器数据输出到读取端口。注意在读取时, 如果读取地址为 0, 那么输出数据为 0, 一般情况下会通过 Verilog 的三目运算符实现这一需求, 我这里采用了位操作, 节省电路开销。

图 2 是寄存器堆的电路图, 可以看到, 通过多路复用器和解复用器, 实现了对寄存器堆的读写操作。读取时, 根据地址选择对应的寄存器输出。写入时, 根据地址选择对应的寄存器写入。

图 3 是本地仿真的波形图, 可以看到, 当写使能信号为 1 时, 数据被成功同步写入到了指定地址的寄存器中。异步读取时, 根据地址输出了对应的数据。

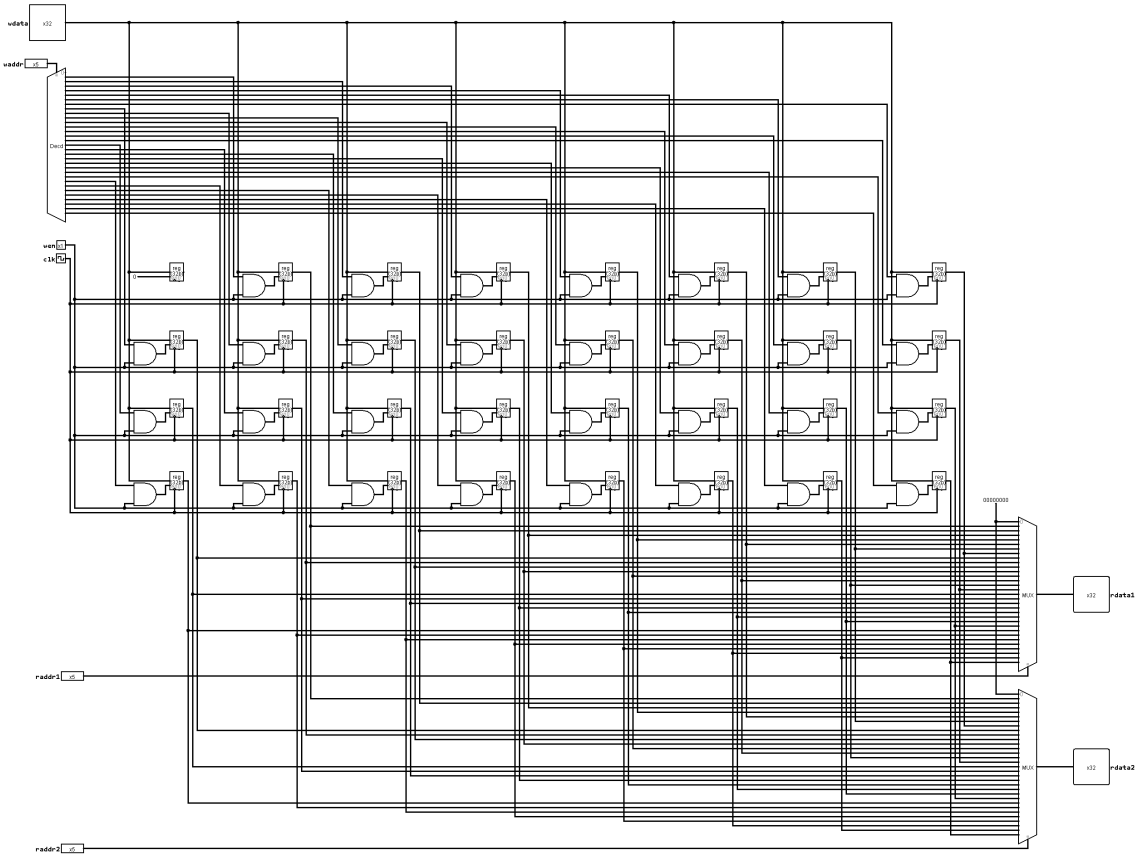


图 2: 寄存器堆电路结构图

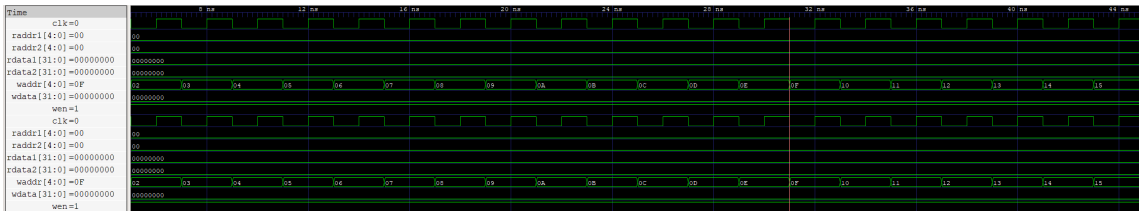


图 3: 寄存器堆波形图

2 ALU 设计

在 ALU 的实验中,我们需要实现五个 ALUop :and, or, add, sub, slt。并且将操作的结果成功表现在 Result, Zero, Overflow, CarryOut 四个输出端口上。

图 4 是 ALU 的关键代码截图。

图中的第六行是 ALU 中唯一一个加法器。通过对数据 B 进行预处理,以及引入 b_invert,可以实现对 B 取相反数的功能,进而可以进行加减法运算。我对 B 取反的方式是位运算,将 b_invert 扩展至 32 位,然后与 B 异或,相比于三目运算符能节省电路开销。

Overflow 表示有符号加减法的溢出,A 和 B 的加减法要区分对待。需要结合 A、B,以及结果的符号位,来判断是否溢出。注意 sub 和 slt 均会用到减法功能。此外,可以通过逻辑表达式化简的知识,将溢出判断的逻辑简化。

CarryOut 表示无符号加法的进位或减法的借位。通过对补码知识的学习,我们可以很容易发现,对于无符号加法,加法器的进位信号 cout 即为 Carryout 信号;对于无符号减法,加法器的进位信号 cout 于 Carryout 信号恰好相反。只要发现这一点,我们就可以很容易地实现 Carryout 信号的输出。

```verilog
// for subtraction
assign  B_trans = {`DATA_WIDTH{b_invert}} ^ B;
assign b_invert = op_sub || op_slt;

// the only adder, for ADD, SUB, SLT
assign {cout, S} = A + B_trans + b_invert;

// Status Flags
assign Overflow = (op_add && (sign_A == sign_B) || b_invert && (sign_A != sign_B)) && (sign_A != sign_S);
assign CarryOut = op_add && cout || op_sub && !cout;
assign     Zero = !Result;
assign   Result = {`DATA_WIDTH{op_and}} & (A & B)
                | {`DATA_WIDTH{op_or}} & (A | B)
                | {`DATA_WIDTH{op_add || op_sub}} & S
                | {op_slt && (sign_S != Overflow)};
```

图 4： ALU 关键代码截图

Zero 表示 ALU 的结果是否为 0。通过对结果逻辑取反即可。

Result 是 ALU 的计算结果。通过对 ALUop 的选择，可以实现不同的运算。按位与和按位或可以很容易实现，加法和减法的实现方法也已经叙述。对于 slt 操作，我们同样是借助减法器，通过判断结果的符号位来判断 A 是否小于 B。但是考虑到溢出的影响，最终需要通过结果符号位和 Overflow 信号的结合来判断。

因为 ALU 中没有涉及多级判断的情况，所有和条件有关的地方我都用位运算实现，多路并行。尤其是 Result 中，只有最多三级门电路，相比于三目运算符生成的多级 MUX 串行计算，大大提高了执行效率。

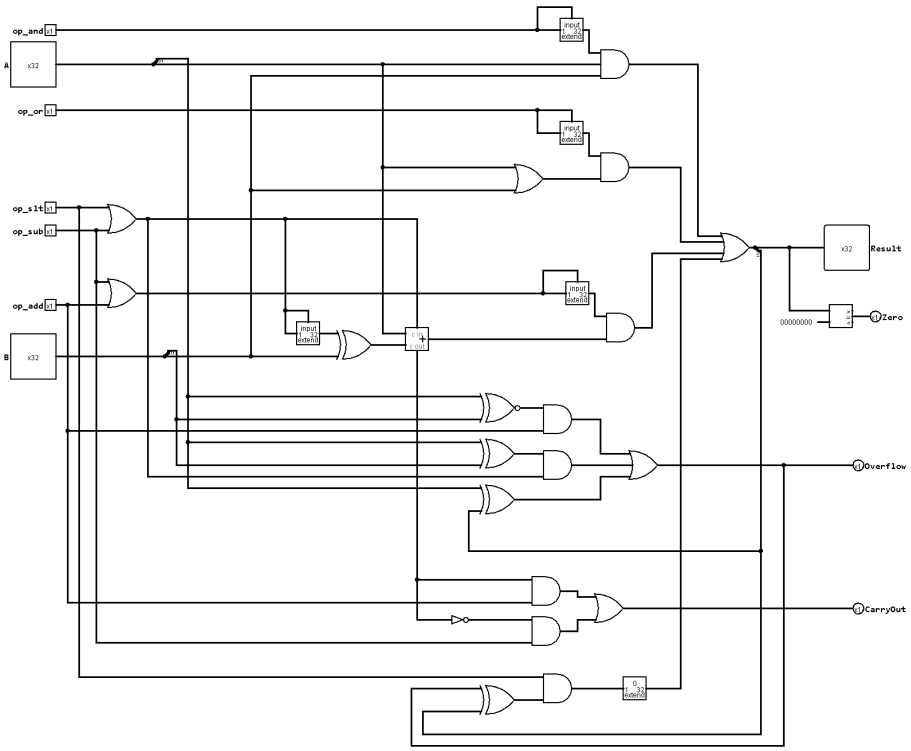


图 5： ALU 电路结构图

图 5 是 ALU 的电路图，可以看到，通过逻辑门，成功使得电路大大简化。

图 6 是本地仿真的波形图，可以看到，电路有条不紊地执行，根据不同的 ALUop 成功输出了正确的结果。

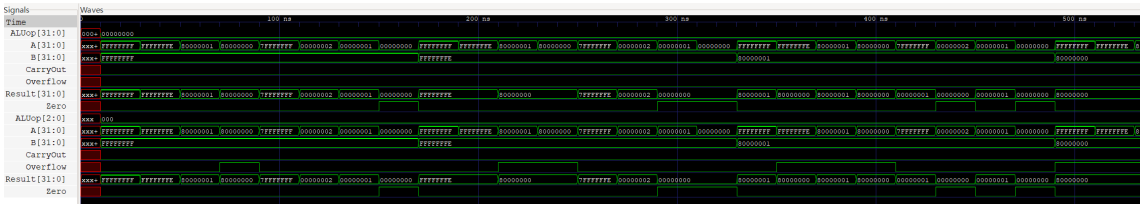


图 6: ALU 波形图

二、 实验过程中遇到的问题、对问题的思考过程及解决方法(比如 RTL 代码中出现的逻辑 bug, 逻辑仿真和 FPGA 调试过程中的难点等)

3 寄存器堆设计问题

在寄存器堆的设计中, 最主要的是 0 号寄存器的的写入和读出。在写入时, 我需要判断写入地址是否为 0, 如果为 0, 那么不进行写入操作。在读出时, 如果读取地址为 0, 那么不管 0 号寄存器中真实数据如何, 都要保证输出数据为 0。这里我采用了位运算的方式, 通过对地址进行位运算, 来判断是否为 0。

4 ALU 设计问题

在 ALU 的设计中, 一个重要的点是溢出和进借位的判断。这里考验我对计算机存储数据方式的认识, 对于同样的二进制数据, 将他看作有符号数和无符号数, 结果是完全不一样的。溢出的判断需要将数据视作有符号数, 考虑 A 和 B 的符号位, 以及结果的符号位。进借位的判断只需要观察出无符号加减法与加法器 cout 的关系即可。我最开始判断无符号减法用的也是比较 A、B、结果最高位的方法, 这里很感谢仲剑学长用群论的知识帮我简化了逻辑。

另外, 怎么共用一个加法器也是一个问题。我通过对 B 进行预处理, 引入 b_invert, 实现了对 B 取相反数的功能。最开始我直接对 B 按位取反 + 1, 殊不知这样又引入了一个新的加法器, 正确的做法是借助加法器来自低位的 1 bit 进位信号, 将 B 取反的过程与 + 1 的过程分开, 再输入到加法器中。从加法器这里我深刻认识到软硬件设计的差别。这里很感谢吉骏雄学长的指导。

5 其他问题

起初, 我没有认真阅读 hello-SERVE 文档, 导致一直用云端模拟的方法检测结果。后来才发现可以直接在本地利用 make 指令完成行为仿真, 整个过程只需要一两秒。本地仿真也可以在 sim_out 文件夹中查看波形图, 相比于将仓库同步到云端, 等待云端动辄好几分钟的模拟时间, 然后下载波形图, 本地仿真的效率提高了很多。我也更深刻认识到熟练使用工具对开发的重要性。

三、 实验所耗时间

在课后, 你花费了大约 10 小时完成此次实验。